RippleKG: Evidence-Aware Incremental View Maintenance for LLM-Generated Knowledge Graphs

Yu-Yi Jhang (R14921039), Ching-I Huang (R14946015), Yu-Lin Chu (R14946003), Ting-Ju Lin (R13921094)
Group 1, Database Management Systems (DB114), National Taiwan University

Abstract—Retrieval-augmented generation brought semantic search into the database; this project brings incremental view maintenance (IVM) into LLM-generated knowledge graphs (KGs). When a source corpus is edited, an LLM-derived KG must be updated. Re-extracting the whole graph is expensive, and invalidating every object reachable from a changed sentence over-invalidates the graph. RippleKG instead treats the KG as a materialized view over sentence-level provenance and propagates a corpus edit only along the evidence that actually changed. We contribute three mechanisms, all materialized in a single ArangoDB instance: M1, a semantic evidence delta that classifies each affected evidence record as added, removed, or unchanged using a canonical triple key; M2, a cost-aware invalidation policy that chooses Skip/Patch/Rebuild per affected object; and persisted per-object freshness state. On Re-DocRED, the incrementally maintained KG matches a from-scratch recomputation over active provenance (0 mismatches over 1,925 relations and 1,078 entities). In a 50-document mixed-edit benchmark, RippleKG reduces affected-evidence maintenance cost by 88.24%, costs 65.13× less than document-level rebuild and 3179.15× less than whole-KG rebuild; in scaling to 500 documents, the whole-KG rebuild gap grows to 27550.36×.

Index Terms—incremental view maintenance, knowledge graph, provenance, retrieval-augmented generation, ArangoDB

I. Introduction

Large language models (LLMs) are increasingly used to build knowledge graphs from text corpora, and graph-based retrieval (GraphRAG) consumes those graphs downstream. In any persistent deployment the source corpus keeps changing: a sentence is corrected, a fact is replaced, a passage is rewritten. The derived KG must follow. Two strategies dominate practice and both are unsatisfactory. A full rebuild re-extracts the entire KG after every edit and is the most expensive option. Naive or generic-traversal invalidation marks every object reachable from a changed sentence as stale; it is cheap to decide but over-invalidates, because graph reachability is far coarser than the evidence that a specific edit actually touches. Existing incremental GraphRAG systems (e.g., LightRAG [4], Microsoft GraphRAG [3], iText2KG [5]) are append/merge oriented: they grow the graph but rarely delete, invalidate, or expose freshness.

We observe that this is exactly the problem incremental view maintenance solves in databases, recast for LLM-generated views. The KG is a materialized view; sentences are base tuples; provenance edges are the dependency graph; a corpus edit is a base-table update that must propagate to derived objects. Our one-line framing:

> RAG brought semantic search into the database;
> RippleKG brings incremental view maintenance into LLM-generated knowledge graphs.

Contributions. (1) We frame LLM-KG maintenance as IVM and identify why classic IVM does not directly apply (Sec. II). (2) We design and implement three mechanisms—M1 semantic evidence delta, M2 cost-aware Skip/Patch/Rebuild policy, and persisted freshness (Sec. IV)—entirely on ArangoDB, so that evidence deltas, invalidation decisions, and freshness are durable database state, not transient objects (Sec. V). (3) We evaluate correctness, over-invalidation, and cost against full-rebuild, generic-traversal, and naive baselines, and report a retrieval/gate/end-to-end study with an independently audited label set (Sec. VI).

TABLE I
Classic IVM assumptions vs. LLM-generated views

Classic IVM assumes	LLM-generated view reality
View is deterministic	Re-extraction paraphrases identical meaning
Dependencies are exact	Dependencies are fuzzy / semantic
Refresh is cheap (eager)	Extraction/judging is costly; refresh must be selective

II. Why Classic IVM Is Insufficient

Classic IVM assumes a view is a deterministic, exact function of its base data and that refresh is cheap enough to run eagerly. LLM-generated views break all three assumptions, as summarized in Table I. Re-extraction may emit textually different evidence for the same underlying claim (paraphrase), so equality must be semantic, not byte-exact. Dependencies are fuzzy and mediated by provenance rather than exact column references. And refresh can be expensive (an LLM call), so it must be selective and, ideally, deferrable. These gaps motivate a semantic delta operator, a cost-aware policy, and visible freshness state.

III. Related Work

RippleKG sits at the intersection of four neighboring lines of work. Incremental LLM-KG / GraphRAG

systems [3]–[5] build and grow KGs from text. Microsoft GraphRAG [3] rebuilds community summaries, LightRAG [4] performs dual-level incremental insertion, and iText2KG [5] merges newly extracted triples into an existing graph. All are append/merge oriented: they add or reconcile nodes but rarely retract evidence, invalidate stale objects, or expose per-object freshness, so the corpus→KG maintenance loop is left open. Classic database IVM [7]–[9] maintains materialized views under deltas that are exact, deterministic, and cheap enough for eager refresh—assumptions Sec. II shows do not hold for an LLM-generated view, motivating a semantic delta and a cost-aware policy rather than algebraic delta rules. KG provenance and outdated-fact detection track which source supports which fact and flag staleness, but stop short of an end-to-end, cost-aware refresh policy backed by persistent decision state. Finally, work sharing the "ripple" name, notably RippleEdits [6], studies ripple effects of editing model weights in an LLM, an orthogonal problem to maintaining a database-materialized KG. To our knowledge, the combination of a semantic evidence delta, a cost-aware Skip/Patch/Rebuild policy, and persisted per-object freshness for a database-materialized LLM-KG is unoccupied.

IV. Method

A. Pipeline and the synchronous spine

A corpus edit is supplied as an EditOp= $(\text{doc\_id}, \text{sent\_idx}, \text{new\_text}, \text{intended\_triples})$. We adopt authored intended triples (the set of triples the edited sentence should support afterwards) as the primary evidence source, so that extraction quality is not the object of study; a lightweight LLM extractor can produce the same interface. Processing follows six stages:

```
corpus edit
 -> changed sentence (text_hash)
 -> affected evidence (1-hop provenance)
 -> M1 semantic evidence delta
 -> affected entity / relation
 -> M2 SKIP / PATCH / REBUILD
 -> persisted KG / provenance / freshness
```

Stages 1–5 form a synchronous spine executed inside one ArangoDB stream transaction: the sentence text, provenance edges, and the existence of entities and relations are always made consistent atomically. Only the derived-aggregate repair (recomputing evidence counts, dropping empty relations, flipping freshness back) may be deferred (Sec. IV-D).

B. M1: semantic evidence delta

Given a changed sentence, M1 collects its old active evidence by a 1-hop outbound traversal over the provenance edges and compares it to the new intended_triples. Relation evidence is keyed by the canonical triple $(\text{head}, \text{rel\_type}, \text{tail})$ after name normalization; mention evidence is keyed by the normalized entity name:

$$\text{new} \setminus \text{old} \to \text{added},\ \text{old} \setminus \text{new} \to \text{removed},\ \text{old} \cap \text{new} \to \text{unchanged}$$

TABLE II
M2 decision table

Evidence delta on object	Decision	Cost
Triple unchanged (paraphrase)	Skip	0
Evidence added	Patch	1
Evidence removed, others remain active	Patch	1
Last active evidence removed	Rebuild	5

The crucial case is unchanged: when only the surface wording changes but the canonical triples are identical (a paraphrase), M1 reports no change—which exact, hash-based IVM cannot do. M1 simultaneously updates provenance edges (insert for added, mark status=removed for removed, leave unchanged untouched) and persists each change to the evidence_deltas collection.

C. M2: cost-aware invalidation policy

For each affected object M2 consults the decision table in Table II. The policy is the cost-awareness: Skip when nothing changed, Patch when an aggregate can be repaired in place, and Rebuild only when the last supporting evidence disappears. Nominal costs are Skip = 0, Patch = 1 (a database write), and Rebuild = 5 (modeling re-resolution/re-extraction as 5× a patch). Each decision is written to refresh_decisions with status=pending.

D. Freshness and refresh execution

Every entity and relation carries freshness_status (fresh/stale) and last_changed_step; history is reconstructable from the delta/decision logs (no bitemporal versioning). apply_refreshes processes pending decisions in two modes. Immediate (default) refreshes within the step, so all objects return to fresh; deferred leaves chosen decisions pending until a later maintenance tick, making staleness visible across steps. Deferral is safe by three invariants: (i) the next edit's M1 does not depend on aggregates; (ii) apply_refreshes is idempotent (it always recomputes from current evidence); and (iii) stale objects are explicitly flagged, i.e., known not-yet-refreshed rather than silently wrong. Queries may request fresh_only to exclude stale objects.

V. System Design and Schema

RippleKG is implemented on a single ArangoDB database; a FastAPI service imports the package as a library so the demo, scripts, and notebook all call the same run_edit path and never drift from the real pipeline. Because we do not modify storage internals, the contribution is framed as logical database design: a provenance schema, evidenced records, AQL provenance traversal, transactional invalidation, and visible freshness. Table III maps each pipeline element to its DBMS concept.

The store uses eight collections: four document collections (documents, sentences, entities, relations), two provenance edge collections along which the ripple travels (mentions: sentence→entity; sentence_supports_relation:

TABLE III
Pipeline→DBMS concept mapping

RippleKG element	DBMS concept
corpus update	base-table update
changed sentence / diff	change capture
provenance edges	forward dependency tracking
evidence delta (M1)	view-level change detection
affected entity/relation	derived-table propagation
Skip/Patch/Rebuild (M2)	cost-aware view maintenance
freshness_status	materialized-view freshness

sentence→relation), and two maintenance collections (evidence_deltas for M1 output and refresh_decisions for M2 output). Because provenance is a real graph, the affected set is exactly a 1-hop outbound traversal from the changed sentence—no multi-hop reachability is needed—which is the structural reason RippleKG avoids over-invalidation.

VI. Evaluation

A. Setup

We build the initial T_0 graph directly from Re-DocRED [2] annotations (a deterministic corpus→KG pair derived from DocRED [1]) without running an LLM: sents become sentences, vertexSet becomes entities and mention edges, labels become relations, and labels.evidence becomes supporting edges. Synthetic edits carry authored intended_triples. The store is ArangoDB 3.12; embeddings (optional) use all-MiniLM-L6-v2 [10].

B. Case studies

Two representative single-sentence edits exercise the policy paths (Table IV). doc0's opening sentence states that Schneider was born in Mediaş, Transylvania, and is a German skeleton racer. In Case A we reorder it into "Born on 13 March 1963 in Mediaş…, Schneider is a German skeleton racer": the canonical triples are identical, so M1 returns only Unchanged deltas, all 12 affected objects Skip, and the incremental cost is zero against a full-document rebuild of nominal cost 60. In Case B we rewrite the birthplace and nationality to Kraków, Poland and Polish; M1 reports the old place-of-birth and citizenship triples removed and the new ones added (date of birth Unchanged), and M2 Rebuilds the relations whose last supporting evidence vanished while Patching those that retain evidence—touching only the affected objects at ∼49% of a full-document rebuild. Crucially, Case A is exactly where reachability baselines over-invalidate: they would mark all 12 mentioned objects stale, whereas evidence-aware comparison correctly skips them.

C. Baselines: correctness, over-invalidation, cost

We compare against three sanity baselines: B0 full rebuild (recompute each object's aggregate from active evidence), B1 generic-traversal invalidation (mark reachable objects stale), and B2 naive invalidation (mark every mentioned object stale). We also report two coarser rebuild granularities: document-level rebuild, which approximates a GraphRAG-style changed-document refresh, and whole-KG rebuild, which recomputes the complete corpus view after every edit. Running up to 100 edits per scenario over 50 Re-DocRED documents produced 357 executable mixed edits (Table V).

TABLE IV
Two representative edits on doc0 (A: paraphrase, B: factual change)

	Case A	Case B
M1 (unchanged)	12	3
M1 (added / removed)	0 / 0	4 / 9
M2 Skip/Patch/Rebuild	12 / 0 / 0	3 / 6 / 7
incremental cost	0	41
full-doc rebuild cost	60	80
reduction	100%	48.75%

TABLE V
Baseline comparison on 50 documents and 357 mixed edits. B0 consistency checked 1,925 relations and 1,078 entities.

Metric	Value	vs. ours
M2 decisions Skip/Patch/Rebuild	2120/358/249	—
RippleKG cost	1,603	1×
Affected-evidence full rebuild	13,635	8.51×
Cost reduction vs. affected rebuild	88.24%	—
B2 naive stale / over-invalidated	2,631	—
Document rebuild (GraphRAG-style)	104,405	65.13×
Whole-KG rebuild (B0 granularity)	5,096,185	3179.15×
B0 consistency	0 mismatches	—

Correctness (B0). After all incremental edits, the materialized evidence_count/status of every object equals a from-scratch recomputation over active provenance edges: 0 mismatches across 1,925 relations and 1,078 entities. This is the classic IVM consistency invariant: incremental Skip/Patch/Rebuild does not sacrifice the maintained KG state.

Cost and over-invalidation. RippleKG costs 1,603 units, compared with 13,635 for the affected-evidence all-rebuild reference, an 88.24% reduction. The naive baseline produces 2,631 stale or over-invalidated objects. At coarser rebuild granularities, document-level rebuild costs 104,405 units (65.13× ours) and whole-KG rebuild costs 5,096,185 units (3179.15× ours). Thus, evidence-aware maintenance is cheaper both than an all-rebuild policy over affected evidence and than document/corpus-granularity rebuilds.

D. Scaling, scenarios, and operational overhead

To check whether the advantage persists as the corpus grows, we repeated the mixed benchmark from 5 to 500 documents. Smaller corpora contain fewer eligible synthetic edits; after 100 documents the workload saturates at 400 executable edits. Table VI reports the representative larger settings. The RippleKG cost remains tied to changed sentence evidence, while whole-KG rebuild

TABLE VI
Scaling with corpus size. Smaller runs (5–25 docs) are omitted from the paper table because they contain fewer executable edits.

Docs	Edits	Ours	Doc rebuild	Whole-KG / ours
50	357	1,603	104,405	3179.15×
100	400	1,882	116,370	5950.21×
200	400	1,957	124,600	11528.89×
500	400	1,957	124,600	27550.36×

TABLE VII
Scenario-level M1/M2 behavior on the 50-document mixed benchmark

Scenario	Skip	Patch	Rebuild	Cost	Naive stale
change tail	400	170	42	380	498
no relation evidence	216	1	15	76	232
remove relation	506	187	189	1,132	900
semantic no-op	998	0	3	15	1,001

TABLE VIII
Runtime, durable maintenance logs, and consistency check

Metric	Value
Runtime	98.23 s
Runtime per edit	275.16 ms
Evidence-delta records	2,727
Refresh-decision records	2,727
Log records per edit	15.28
Checked relations / entities	1,925 / 1,078
B0 mismatches	0

TABLE IX
End-to-end accuracy (20 should-edit operations)

Stage	before verifier	after
EditOp content accuracy	80%	95%
M1 added/removed accuracy	80%	100%
End-to-end M2 accuracy	80%	100%
M2 policy-rule accuracy	100%	100%

increases with corpus size: at 500 documents, whole-KG rebuild is 27550.36× more expensive than RippleKG.

The scenario breakdown in Table VII explains where the savings come from. For semantic_noop, M1 classifies almost all evidence as unchanged (998 records), so M2 skips nearly every affected object: RippleKG costs only 15 units while the naive baseline marks 1,001 objects stale. In contrast, remove_relation requires many Patch and Rebuild decisions because evidence actually disappears. RippleKG therefore avoids work selectively rather than blindly.

Finally, we measured wall-clock overhead on the 50-document mixed benchmark (Table VIII). RippleKG processed 357 edits in 98.23 seconds (275.16 ms/edit) while persisting 2,727 evidence-delta records and 2,727 refresh-decision records, or 15.28 durable maintenance-log records per edit. This supplements the nominal cost model with an operational measurement.

E. Retrieval, relevance gate, and end-to-end

A hybrid front end can select which sentences to edit before M1/M2 run. On a 50-fact evidence-retrieval benchmark, Transformer retrieval reaches Hit@1 = 86%, Hit@3 = 100%, MRR = 0.93; graph provenance gives a high-recall (R= 100%) but lower-precision (P= 58.8%) affected set. An LLM relevance gate raises precision to 76.9% at R= 100% (F1 = 87%). End-to-end, after adding a deterministic replacement-aware verifier in front of M1, EditOp content accuracy is 95% and M1 added/removed accuracy and end-to-end M2 accuracy both reach 100% (Table IX); M2 policy-rule accuracy is 100% throughout, confirming that residual end-to-end error originated in the intended-triple verifier, not the policy.

F. Label audit

The should-edit gold set (30 replacement cases, 150 candidate sentences) was independently re-checked against its annotation rule: of 150 labels, 0 were overturned; all 20 True labels genuinely assert the old fact, and 12 borderline False labels (the value appears in the sentence but is not asserted, e.g. "the Canadian province of Alberta") were confirmed correct. The audit notably validates that the same word in one sentence is labeled True for the fact it asserts and False for a fact it merely mentions.

VII. Limitations

Evidence is supplied as authored triples (option A); an LLM extractor (option B) shares the same interface but is left to future work. The main cost comparison uses nominal Skip/Patch/Rebuild units, although we now also report wall-clock runtime and durable maintenance-log counts; dollar/token costs for a deployed LLM extractor are not measured. The benchmark is larger than the initial case study (up to 500 documents and 400 executable edits), but it is still synthetic and centered on replacement-style updates; some synthetic replacements are semantically unnatural. Query-time lazy refresh, max_staleness, and entity-resolution ripple are designed but not implemented.

VIII. Conclusion

RippleKG reframes the maintenance of an LLM-generated knowledge graph as incremental view maintenance and realizes it entirely as durable ArangoDB state: a semantic evidence delta (M1), a cost-aware Skip/Patch/Rebuild policy (M2), and persisted per-object freshness. A corpus edit ripples only along the evidence that actually changed, yielding a KG consistent with full recomputation over active provenance while marking far fewer objects stale and costing orders of magnitude less. RAG brought semantic search into the database; RippleKG brings incremental view maintenance into LLM-generated knowledge graphs.

IX. Member Contributions

The division of labor follows the ownership structure fixed in the project proposal. The percentages reflect each member's overall effort and sum to 100%.

TABLE X
Per-member contribution

Member	Primary contributions	%
Yu-Yi Jhang	Re-DocRED loader and T_0 build, synthetic edit set, B0/B1/B2 baselines	25
Ching-I Huang	ArangoDB schema (8 collections), indexes and provenance persistence, AQL 1-hop traversal and DB inspection	25
Yu-Lin Chu	M1 semantic evidence delta, M2 cost-aware policy, refresh execution	25
Ting-Ju Lin	Evaluation harness and metrics, label audit, report, slides, demo and video	25
Total		100

References

[1] Y. Yao et al., "DocRED: A Large-Scale Document-Level Relation Extraction Dataset," in ACL, 2019.
[2] Q. Tan et al., "Revisiting DocRED—Addressing the False Negative Problem in Relation Extraction," in EMNLP, 2022.
[3] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130, 2024.
[4] Z. Guo et al., "LightRAG: Simple and Fast Retrieval-Augmented Generation," arXiv:2410.05779, 2024.
[5] Y. Lairgi et al., "iText2KG: Incremental Knowledge Graphs Construction Using Large Language Models," arXiv:2409.03284, 2024.
[6] R. Cohen et al., "Evaluating the Ripple Effects of Knowledge Editing in Language Models," TACL, 2024.
[7] A. Gupta and I. S. Mumick, "Maintenance of Materialized Views: Problems, Techniques, and Applications," IEEE Data Eng. Bull., 1995.
[8] C. Koch et al., "DBToaster: Higher-order Delta Processing for Dynamic, Frequently Fresh Views," in VLDB, 2014.
[9] F. McSherry et al., "Differential Dataflow," in CIDR, 2013.
[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in EMNLP, 2019.